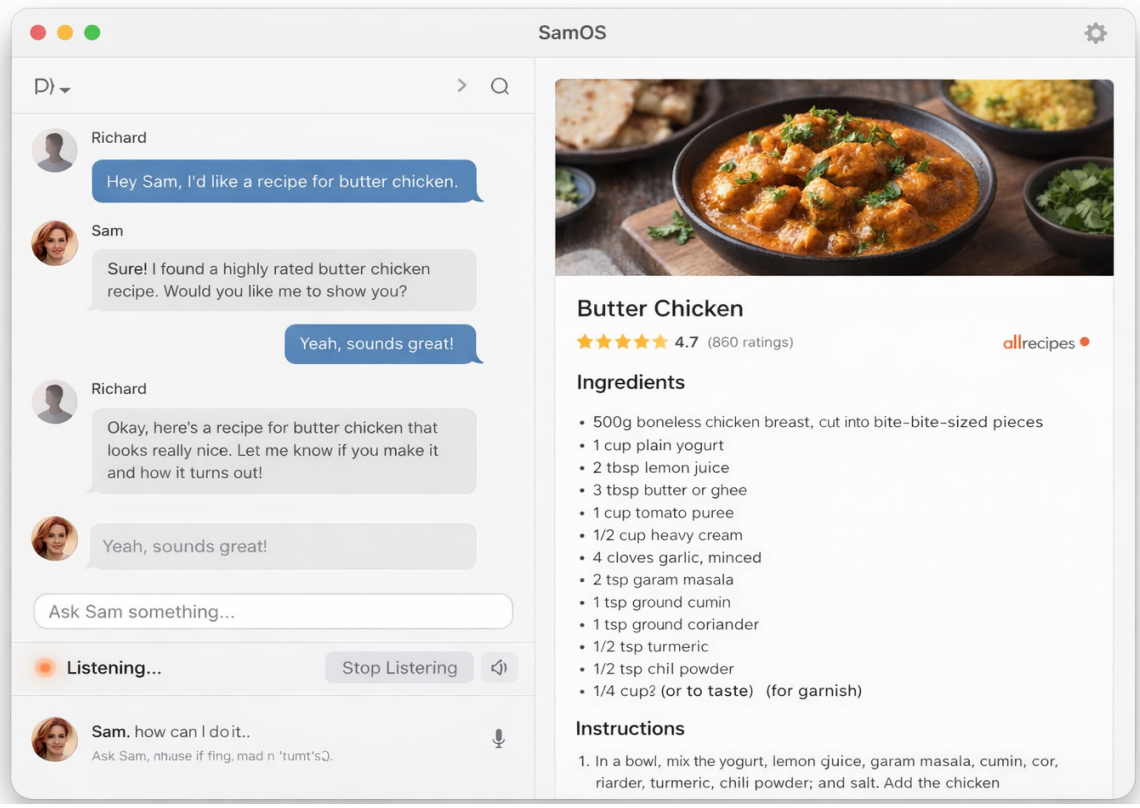
SamOS

Final Specification v1

Date: 2026-02-05



Owner: Richard (personal use)

1. Executive summary

SamOS is a macOS desktop application that behaves like a friendly, voice-first companion: always listening for a wake phrase, holding natural conversations, and showing rich output (text, images, cards) inside its own UI. A local LLM acts as the primary router and conversational brain, and selectively delegates to OpenAI when needed. SamOS stores long-term memory in a local encrypted database and can extend itself via installable capability packages. When a requested capability is missing, SamOS generates a ready-to-paste build task to implement that capability, and (with explicit approval) can install or update capability packages.

Key outcomes in v1

- Always-on wake word ("hey sam") with a clear Stop Listening control
- Speech-to-text (local Whisper), local router LLM (Ollama), optional OpenAI delegation
- ElevenLabs text-to-speech (configurable voice ID)
- Two-pane UI: chat transcript + output canvas (markdown + images + cards)
- Local memory store (SQLite) with view/edit/delete controls
- Capability packages (Python tool servers) installable/updatable with user approval
- Deterministic "capability gap" flow that outputs Claude build instructions

2. Locked-in decisions

Area	Decision
Platform/UI	Native macOS app in SwiftUI (single-window, split view)
Wake word	On-device wake word engine (Porcupine/Picovoice)
STT	Local Whisper (WhisperDesk-style integration)
Primary brain	Local LLM via Ollama (router + friendly conversation)
Delegation	OpenAI used only when router decides it is required
TTS	ElevenLabs (voice configurable by voice_id; no hardcoding)
Memory DB	SQLite with optional at-rest encryption; user-facing memory manager
Capabilities	Installable capability packages as local Python tool servers
Approvals	Explicit approval required for install/update and for each new permission scope
Output	UI blocks (markdown, image, card, list) rendered in output canvas
Extensibility	Missing capability -> generates Claude build prompt + optional installer bundle

Note: voice selection is treated as a user-configurable setting (voice_id).

3. UX and UI specification

3.1 Primary window layout

Single window with a two-pane layout: left pane for the conversational transcript and input; right pane as an Output Canvas where Sam can render rich results (recipes, images, lists, cards).

- Top bar: app title (SamOS) and Settings (gear).
- Left pane: message list (user/assistant bubbles) + text input box.
- Bottom status strip: Listening/Idle/Thinking/Speaking indicator; Stop Listening button; Mute Voice toggle.
- Right pane: Output Canvas that supports markdown, images, and structured cards.
- All assistant actions that produce displayable output must also be visible in the transcript as a short spoken-style message.

3.2 Interaction patterns

- Wake: user says "hey sam"; Sam plays a subtle earcon and begins an active capture session.
- Barge-in: if user starts speaking while Sam is talking, Sam immediately stops TTS playback and listens.
- Confirm-first: for long outputs (recipes, multi-step instructions), Sam asks "Want me to show it?" before rendering.
- Stop Listening: immediately disables wake word engine and microphone capture until re-enabled.
- Capability gap: when Sam cannot perform a request, it explains the limitation and offers a build task to add that capability.

4. System architecture

4.1 Core runtime loop

Always-on wake word runs locally. On trigger, audio is captured into a bounded session and transcribed locally. The local router LLM produces a strict JSON Action. Actions may: (a) respond directly, (b) call a local tool, (c) delegate to OpenAI, or (d) initiate a capability-gap workflow. Tool outputs are rendered as UI blocks and optionally spoken via ElevenLabs.

4.2 High-level components

Component	Responsibility
WakeWordService	Detects "hey sam" on-device; triggers capture session.
AudioCaptureService	Mic capture + pre-roll buffer; silence detection; session boundaries.
STTService	Local transcription (Whisper). Returns text + optional timestamps.
RouterLLMService (Local)	Primary conversational brain and planner; outputs Action JSON.
OpenAIService	Optional delegation for web-heavy tasks or complex reasoning.
ToolsRuntime	Executes tool calls; manages capability packages; returns UI blocks.
TTSService	ElevenLabs speech generation and playback; barge-in stop.
MemoryStore	SQLite storage for memories, summaries, settings; provides retrieval.
UI Renderer	Renders transcript + output canvas from UI blocks.

5. Contracts and schemas

5.1 Router Action JSON (strict)

```
{ "type": "TALK | TOOL | DELEGATE_OPENAI | CAPABILITY_GAP", "say": "string (optional, spoken via TTS)", "tool": { "name": "string", "args": { "any": "json" } }, "delegate": { "task": "string", "context": "string (optional)" }, "gap": { "goal": "string", "missing": "string", "proposed_capability_id": "string (optional)" } }
```

Rules

- The router must return valid JSON and only one top-level action.
- If type=TOOL, tool.name must match a registered tool in the Tool Registry.
- If a tool is missing, ToolsRuntime returns a deterministic NotImplemented result, and the router must switch to CAPABILITY_GAP.
- say is always the short conversational line; long content is rendered in the Output Canvas via UI blocks.

5.2 UI Blocks schema

```
[ { "type": "markdown", "markdown": "string" }, { "type": "image", "url": "string", "alt": "string?" }, { "type": "card", "title": "string", "subtitle": "string?", "fields": [ { "k": "string", "v": "string" } ], "actions": [ { "label": "string", "tool": "string", "args": {} } ] } ]
```

6. Capability packages (installable tools)

Capabilities are installable packages that provide one or more tools. In v1, each capability runs as a local Python tool server (localhost) with a simple JSON-over-HTTP protocol. The Swift app remains the stable host; capabilities can be installed, updated, disabled, or removed at runtime.

6.1 Capability folder layout

```
~/Library/Application Support/SamOS/Capabilities/ images.basic/ manifest.json tools.json
server.py tests/ test_smoke.py
```

6.2 Capability manifest (example)

```
{ "id": "images.basic", "name": "Image Search", "version": "1.0.0", "entrypoint":
"server.py", "permissions": [ "web.read" ], "tools": [ "image_search" ] }
```

6.3 Install/update approval flow

- Sam proposes install/update with a Change Proposal (capability id, version, tools, permissions, file list).
- UI shows Approve / Reject (optional: "Trust this capability" for future updates).
- On Approve: installer writes files only inside Capabilities/; validates manifest/tools schemas; runs smoke test; starts tool server; registers tools.
- All actions are logged in an Audit Log view.

7. Memory and learning (local)

SamOS stores memory locally. It uses a structured memory model to keep context small while still feeling personal. The user can inspect, edit, delete, and export memories at any time.

7.1 Memory types

- Profile facts (stable): preferences, names, routines.
- Episodes (events): short summaries of meaningful moments.
- Mood entries (optional): only saved when explicitly stated or confirmed.

7.2 Storage (SQLite)

```
Tables (v1): - messages(id, ts, role, text) - memories(id, ts, type, title, content,
tags_json, importance, embedding_blob?) - daily_summaries(date, summary_text,
embedding_blob?) - settings(key, value_json)
```

7.3 Retrieval (v2-ready)

- Compute embedding for the latest user utterance (local or delegated).
- Retrieve top-K relevant memories by cosine similarity.
- Inject only retrieved memories into the router prompt as "Relevant memories".

8. Controls, privacy, and safety (practical)

Even as a personal app, SamOS includes clear controls to prevent accidental capture and to keep data local by default.

- Stop Listening: hard disables wake engine and mic capture immediately.
- Listening indicator: always visible (Idle/Listening/Thinking/Speaking).
- Local-first: wake word, STT, router, and memory are local by default.
- Permission scopes per capability: web.read, filesystem.read/write (scoped), exec.local_process.
- Secrets (API keys) stored in macOS Keychain.
- Optional DB encryption for at-rest protection.

9. Build plan (milestones)

Milestone	Includes	Done when
M1 - UI + Contracts	SwiftUI window, transcript, output canvas, Action/Tool contracts	Mock router drives UI blocks; app stable
M2 - Wake + STT	Wake word + audio sessions + Whisper STT	"hey sam" -> transcript appears reliably
M3 - Local router + Tools	Ollama router constrained JSON; show_text/show_image/show_list	Voice-first asks to render correctly
M4 - ElevenLabs TTS	Streaming TTS + barge-in stop	Natural talk + interruption works
M5 - Capability installer	Capability folder, approvals, tool servers	Install/update capability with approval
M6 - Memory manager	SQLite store + UI to view/edit/delete	Memories persist and are user-controlled
M7 - Capability gap -> Claude	Deterministic build prompt generator	Missing tool produces a paste-ready Claude task